AI ACADEMY · LEVEL 2 · GRADE 7 · WEEK 1 · STUDENT LESSON

Precision in Algorithms

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Transparent Book Recommender

THE CONNECTED WEEK

Predict the steps. Run the script. Explain the output.

Translate equal groups into Python, inspect assignment and print, and preserve an honest expected/actual record.

Keep one habit: Read the requirement → trace values → run the saved code → explain the first mismatch.

MAIN LESSON · SECTION 1

From precise paper steps to a runnable script

> Keep predictions and actual runs separate from the first program.

Start with an ordered calculation

A computer follows the saved instructions in order. Start by predicting a small equal-packs calculation, then inspect how Python assigns values and displays output.

Retrieve the evidence habit and equal-group readiness

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Four packs contain six counters each. What steps find and show the total?

Your learning goals

- I can translate a short paper calculation into a Python script.
- I can predict and explain the order of executed statements.
- I can keep expected and observed **output** separate when checking a **program**.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

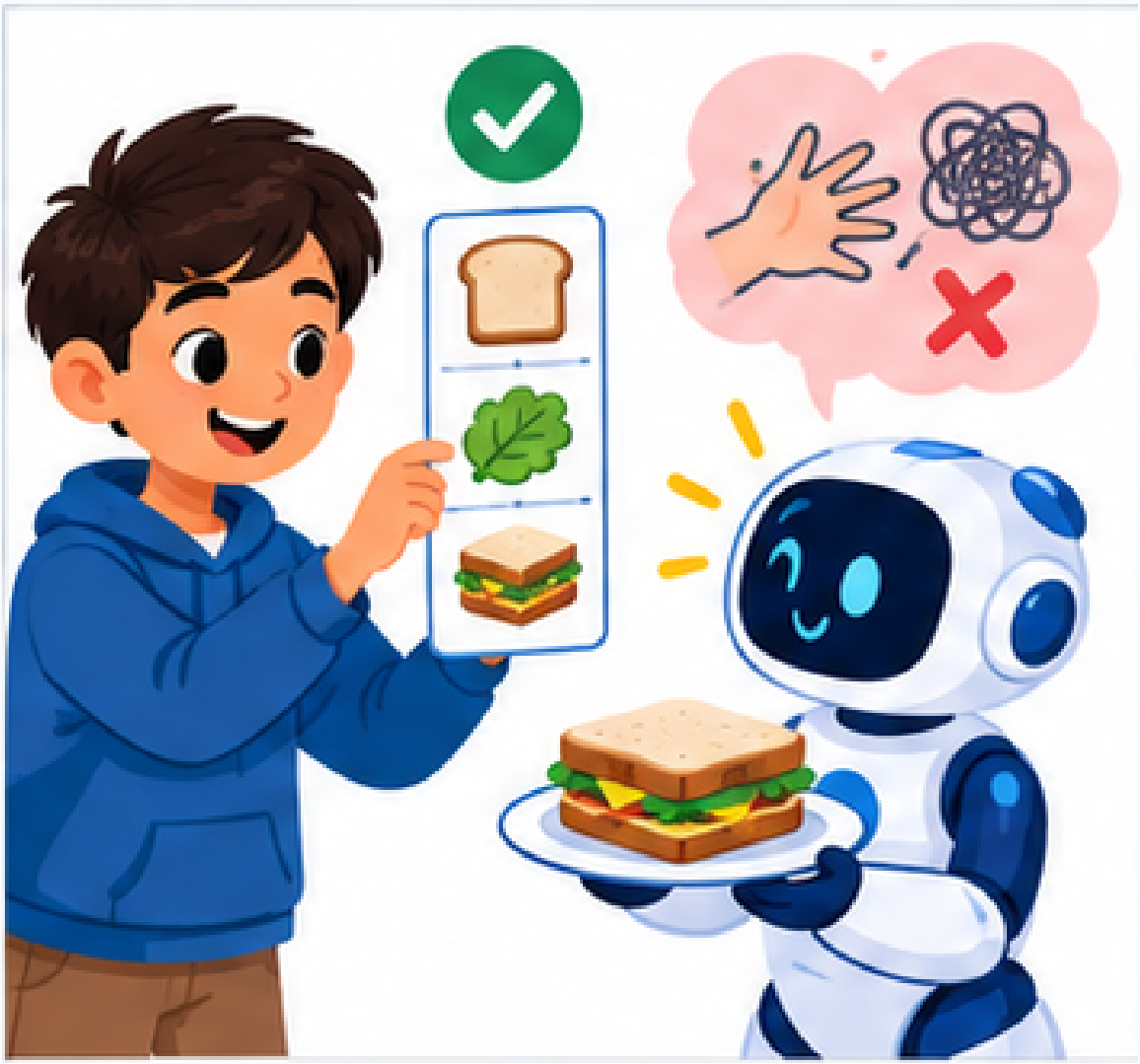
Your learning target

Remember before reading

Without opening the old lesson, explain one key idea from Expo and Defense. Give an example and a mistake that the idea helps you avoid.

Week 1: Precision in **Algorithm**s

Computers follow stated steps, not intended meaning.
</details>



THINKER MISSION

Write two sandwich **algorithm**s; expose hidden **assumption**s.

Thinker name and date:

Write or draw your reasoning and evidence.

Readiness for the connected explanation

This week’s end goal

By the end, I can translate an ordered **algorithm** into Python and check state after each line.

MAIN LESSON · SECTION 2

Prepare your first saved Python program

> Use the browser Python lab or the teacher-prepared editor; a paper trace remains a prediction.

State the counters requirement and scope

In Grade 6 you traced rules on paper. Now you will write a small Python **program** and check what it actually does. Begin with a counter-packing calculation, not the whole activity adviser.

The requirement is to find the total counters in equal packs and display that total. For this first **program**, the supplied quantities are whole, nonnegative numbers; it does not yet validate typed **input**.

Prepare source file, run control and output record

1. Use a teacher-prepared Python 3 editor that can run a saved .py file. Open a new file called counters.py, type the code exactly as shown in the next section, save it, and use the editor’s Run command.

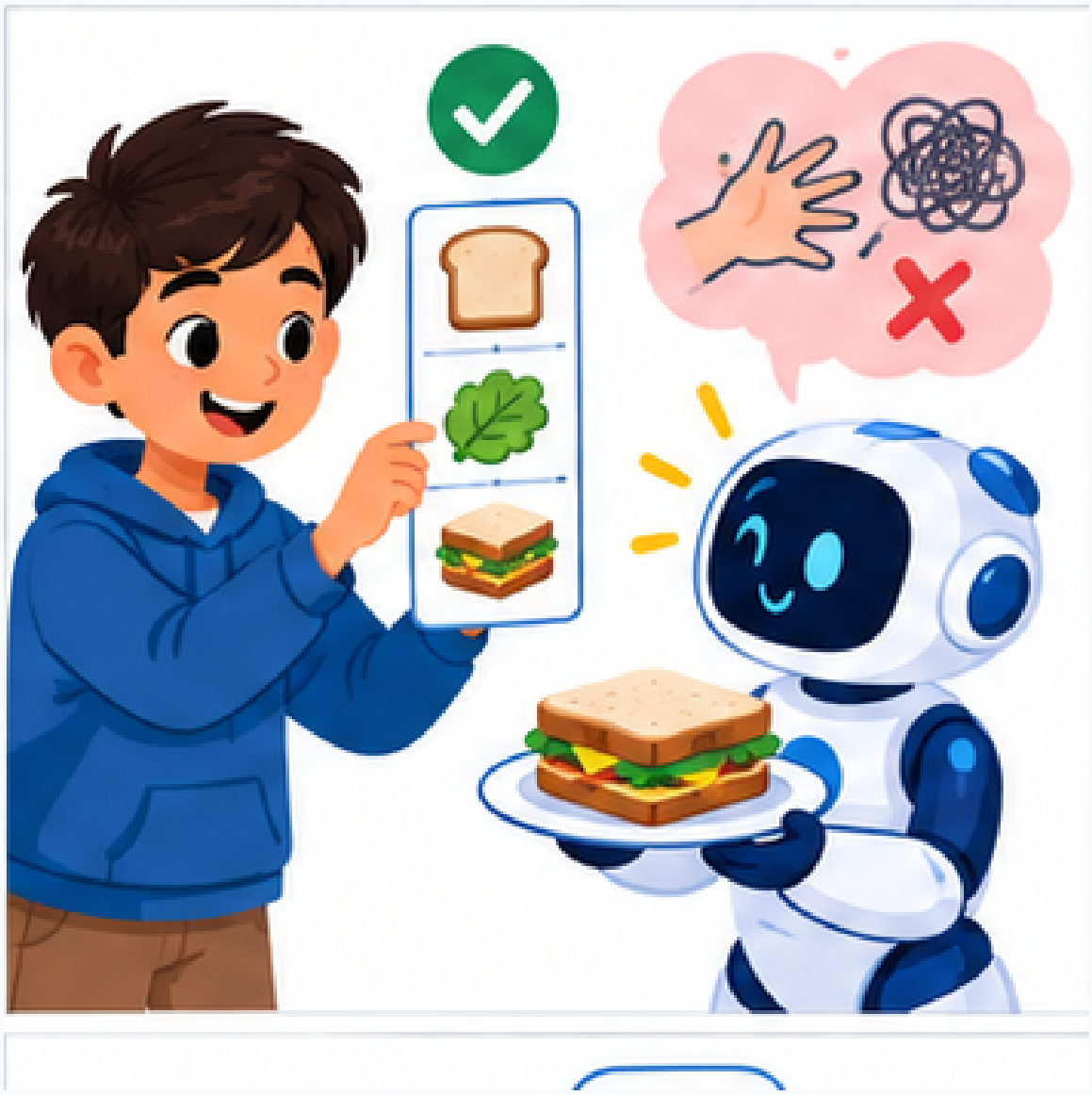
   Read the **output** area separately from the source editor. No account, AI tool, package installation or personal data is needed. If a device is unavailable, trace on paper and label the result predicted rather than executed.

▼ **Earlier wording, alternatives and picture transcripts**

PICTURE STORY

Notice → Predict → Explain

Begin with visible **evidence**. Do not explain more than the picture and information support.



MYSTERY

The learners must use precision in **algorithm**s to solve a problem. What information do they have? What decision or calculation do they make? What would count as **evidence** that it worked?

Three **observation**s (what you can point to):

Write or draw your reasoning and evidence.

My **prediction** and the clue supporting it:

Write or draw your reasoning and evidence.

A closer explanation

MAIN LESSON · SECTION 3

Read each Python statement in order

> Assignment stores a value; print displays it.

Connect paper steps to names and output

A calculation from paper to output

PAPER STEP	PYTHON INSTRUCTION	EFFECT
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

The complete counters.py script

```
packs = 4
per_pack = 6
total = packs * per_pack
print(total)
```

Read assignment, multiplication and print

- Read the script from top to bottom. The equals sign assigns the value on its right to the name on its left. The star means multiplication. print(total) displays the value of total; it does not display the letters in the name.

  Type ordinary straight characters and keep the parentheses. The Python prompt symbols >>> are not part of this saved script.

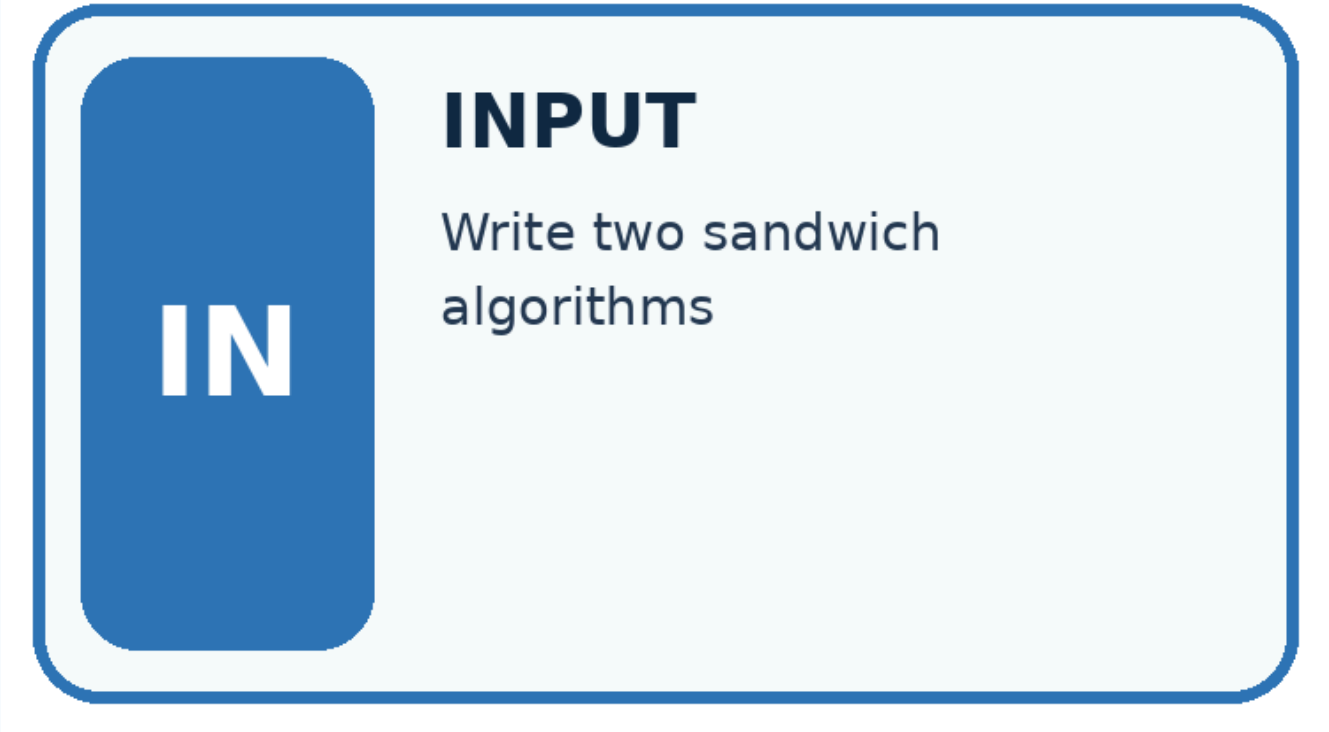
- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

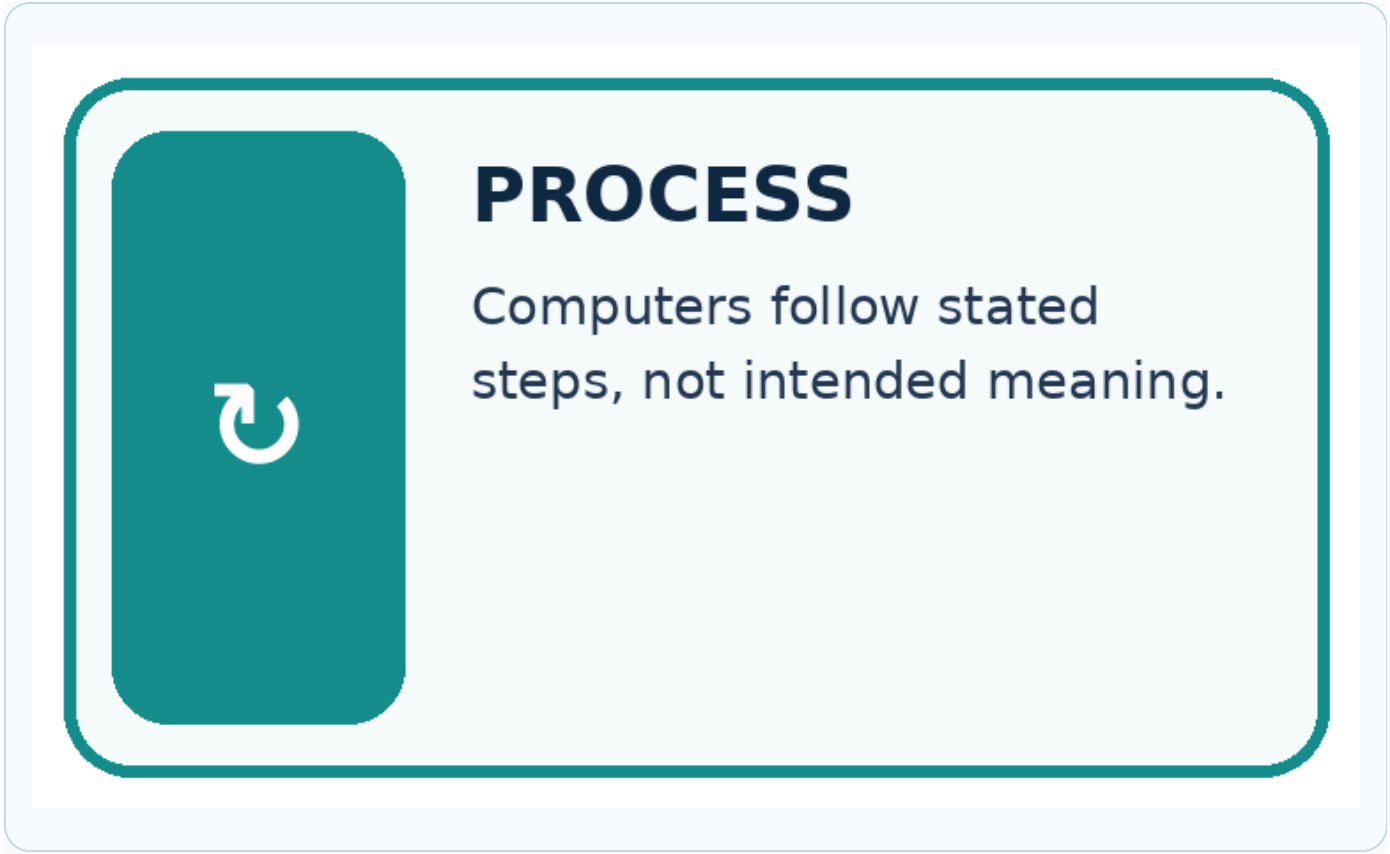
VISUAL **MECHANISM**

Input → **Process** → **Output**

Trace the information. At each stage, ask what changed and how you could test it.



↓


PROCESS
Computers follow stated steps, not intended meaning.
OUTPUT
expose hidden assumptions.
OUT

Which stage could fail even when the other two are correct? Explain:

> **Write or draw your reasoning and evidence.**

A closer explanation

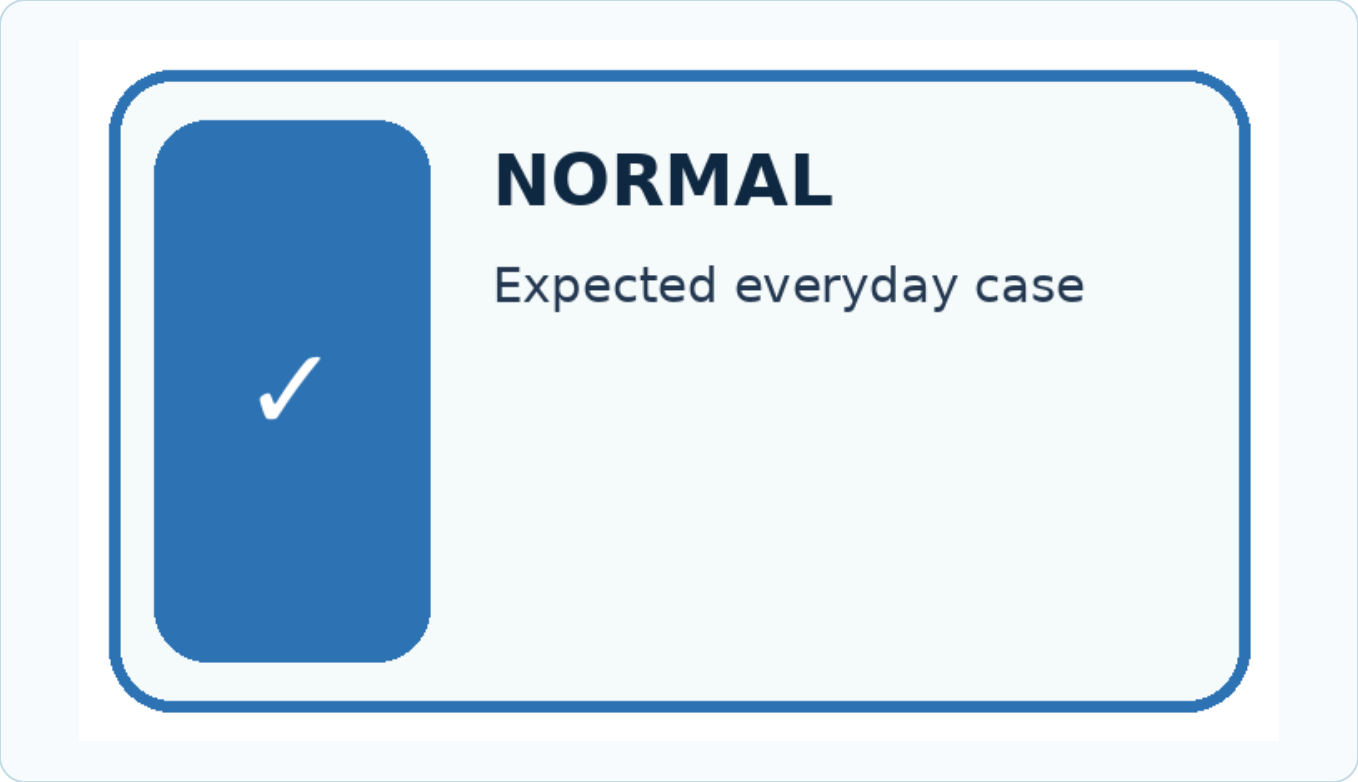
Connect the picture and the explanation

MAIN LESSON · SECTION 4

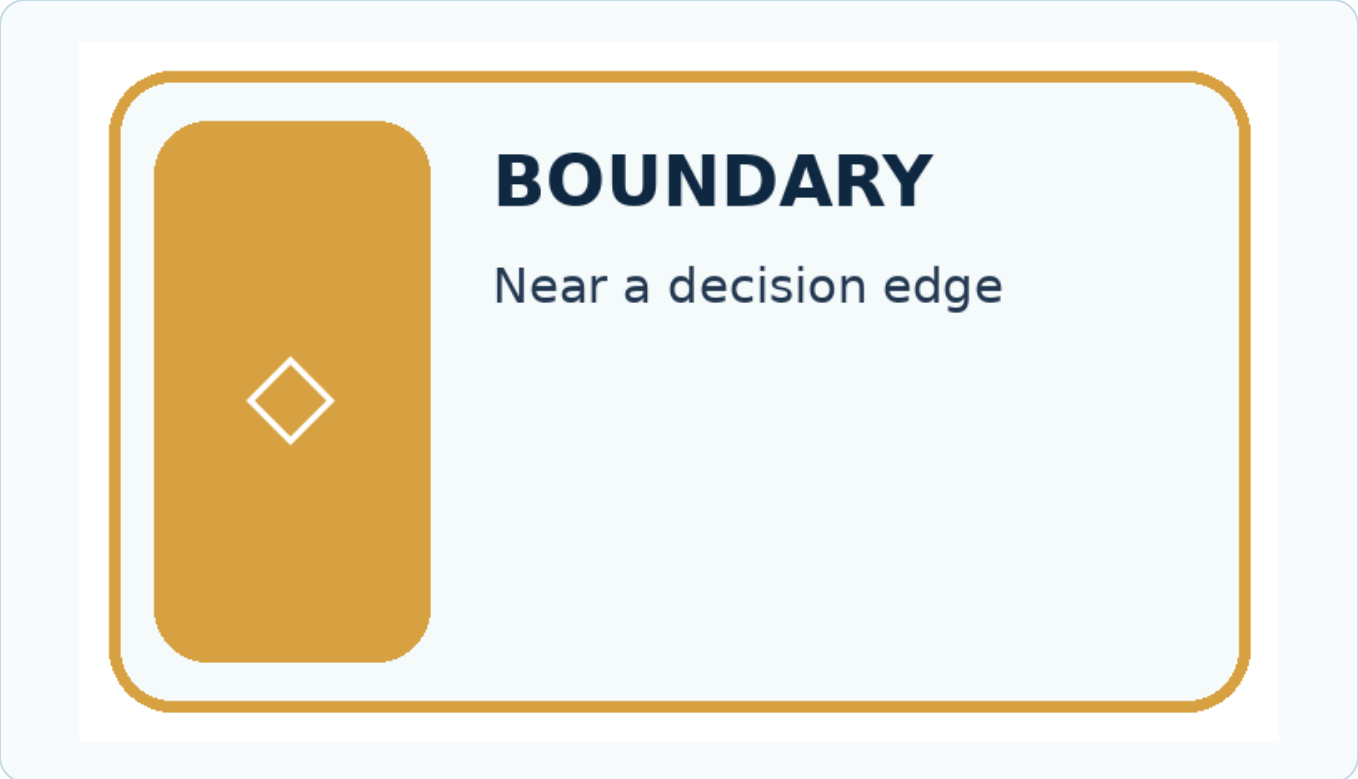
Compare ordinary, boundary, missing and transfer cases

> A clear procedure makes its assumptions and required inputs visible.

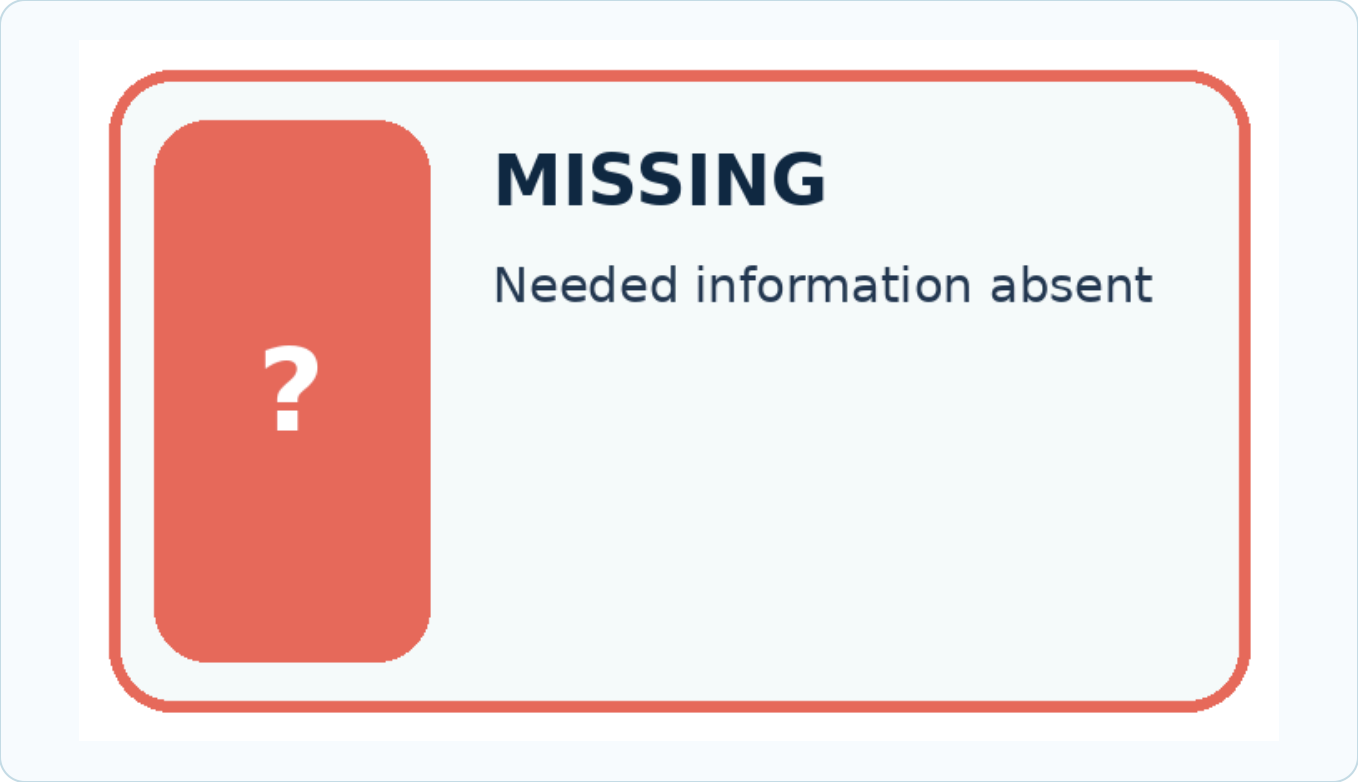
Four pictures with a purpose



Normal case: predict the result and deciding evidence.



Boundary case: identify the exact limit.



Missing information: name what must be clarified.



Transfer case: explain which procedure still applies.

Choose a revealing case

- Which case is most likely to reveal a hidden **assumption**? Why?

> **Write or draw your reasoning and evidence.**

Predict the supplied changed case

- Change: Set boxes=0, keeping the two loose books.

▼ Check after predicting

Prediction: The printed result is 2.

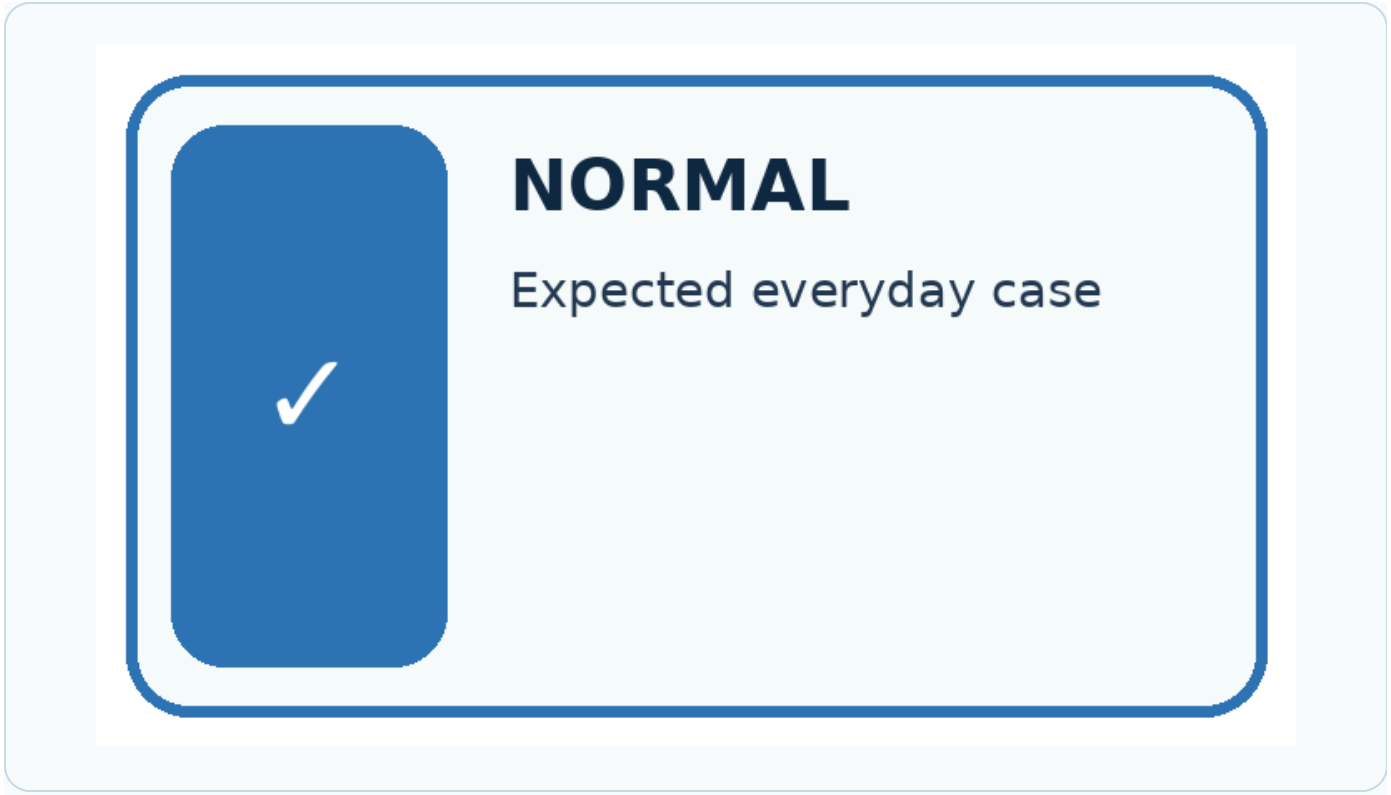
Reason: 0×5+2=2; there is no rule saying zero boxes means zero loose books.

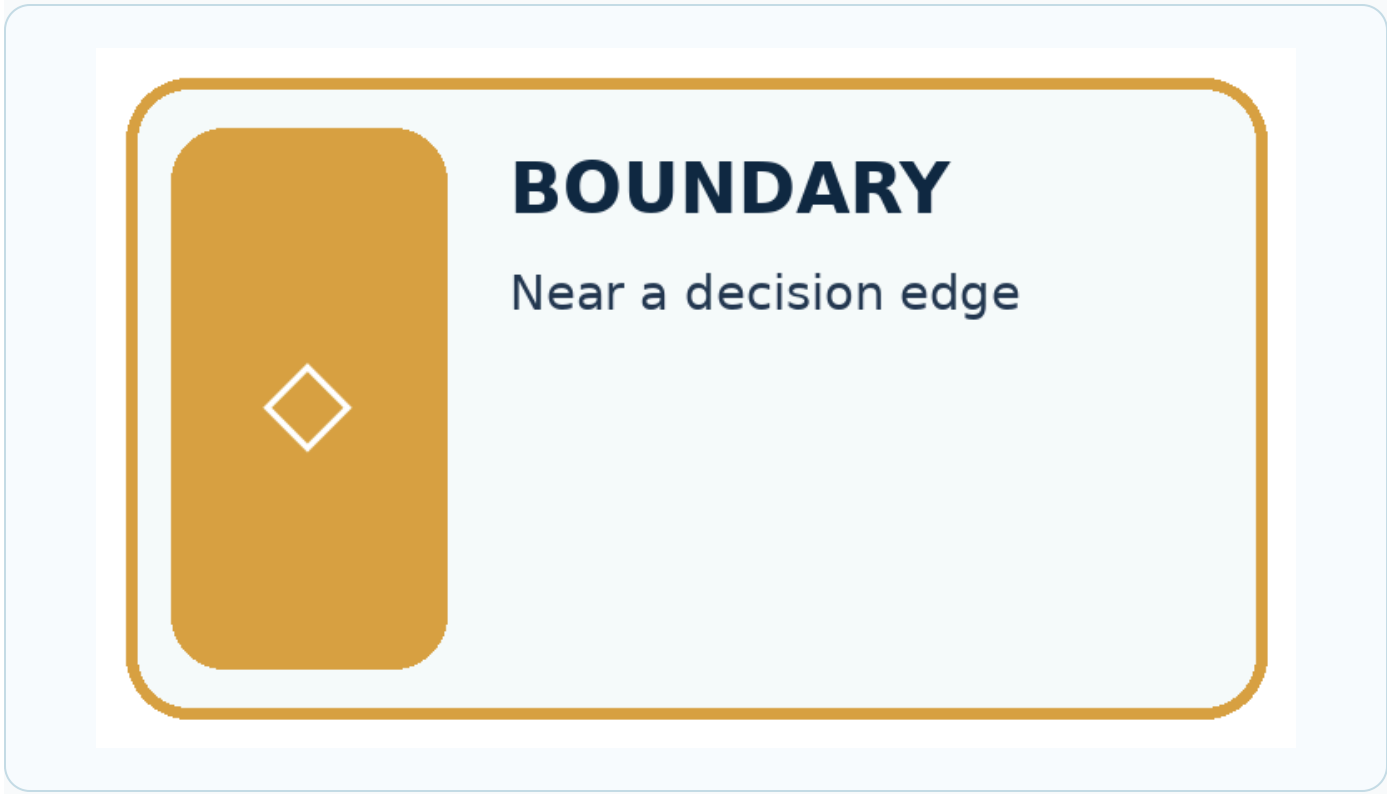
▼ Earlier wording, alternatives and picture transcripts

CASE GALLERY

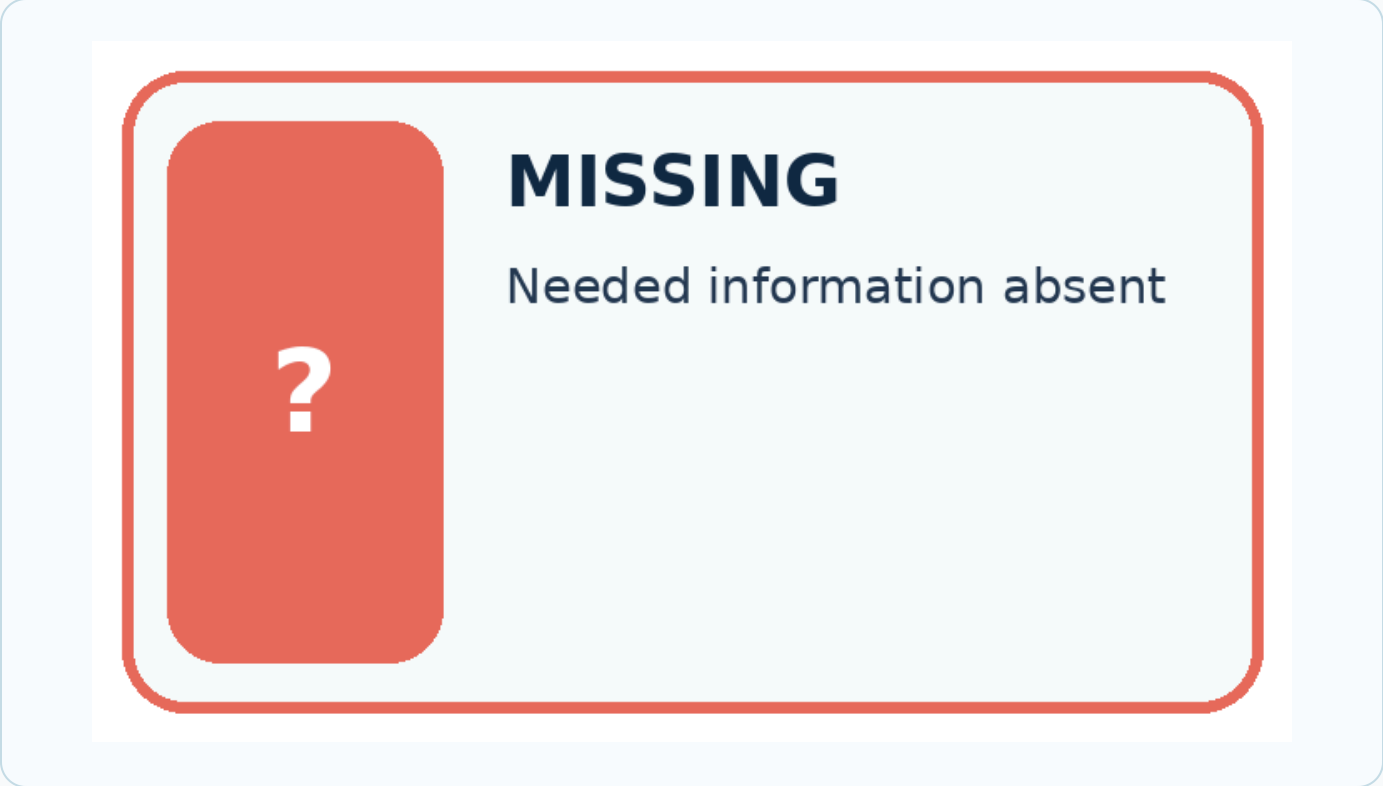
Four Tests, Four Kinds of **Evidence**

A system is not understood until it survives more than one comfortable example.





NORMAL CASE: predict and name **evidence**. BOUNDARY CASE: predict and name **evidence**.





MISSING INFORMATION: predict and name **evidence**. TRANSFER CASE: predict and name **evidence**.

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace counters, then the separate book-box update

> Follow names and displayed output without treating assignment as an equation.

Trace the four-pack program line by line

After line	Names and values	Displayed output
packs = 4	packs is 4	Nothing yet
per_pack = 6	packs is 4; per_pack is 6	Nothing yet
total = packs * per_pack	total is 24	Nothing yet
print(total)	Values are unchanged	24

Write 24 as the expected **output** before running. Then copy the actual **output** into a separate column. A matching result supports this case, not every possible **input**.

A saved file may contain an old edit: save again and rerun the whole script. Do not type the expected number into the **output** record without running.

Check whether a name exists before reading it

Why is there no displayed output after the third line? What changes if print(total) is placed first in a fresh run, before any assignment? Explain using which names exist.

Return to the pack-counting diagram

AI ACADEMY / AI-2/1 · 03 · INSPECT THE LESSON MODEL

A calculation from paper to output

Paper step	Python instruction	Effect
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

Follow the labelled picture with the matching explanation. Open the vector diagram

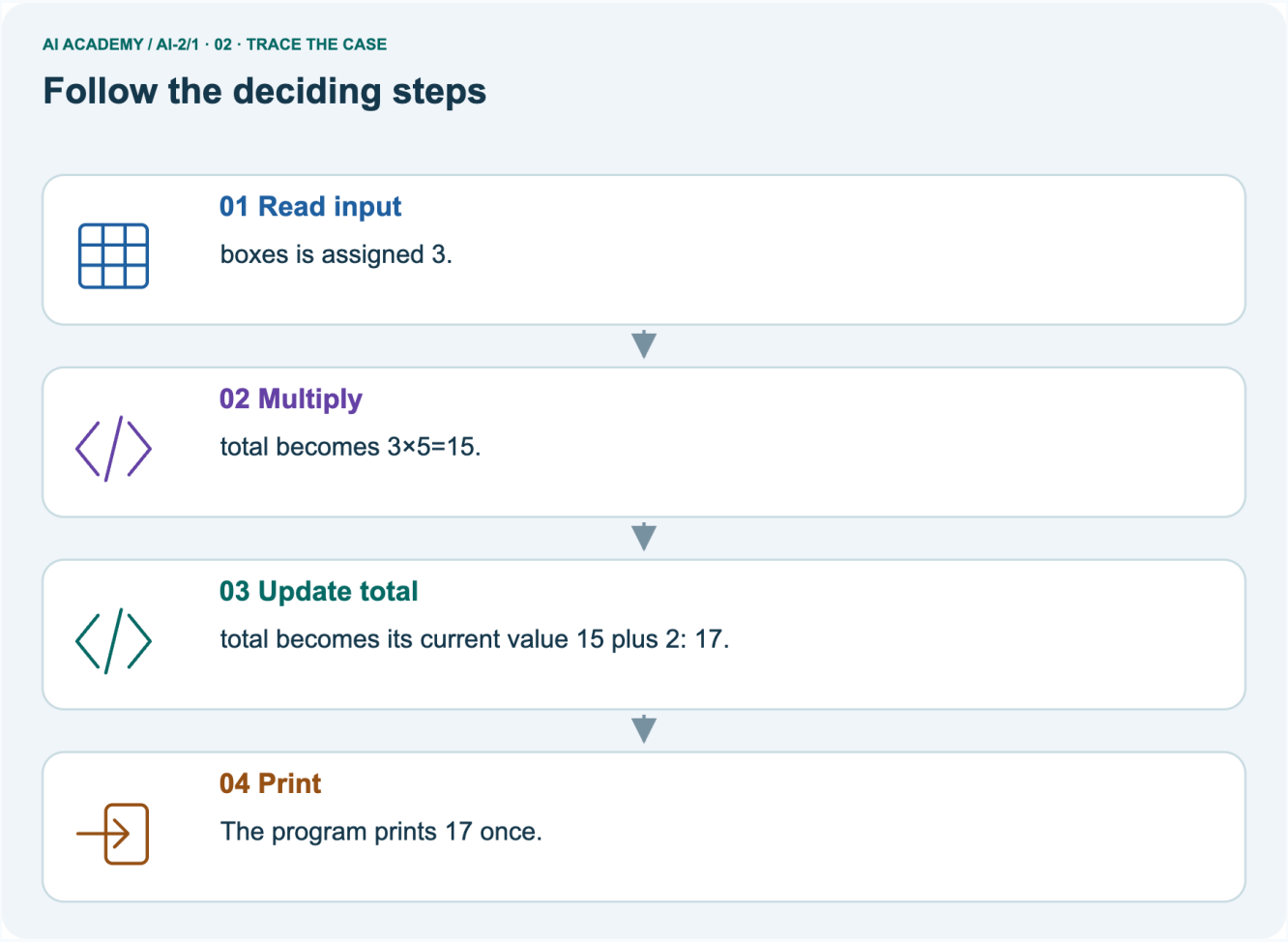
Read the separate book-box facts

A fictional book-box counter has three boxes with five books each and two loose books.

Python instructions: boxes=3; total=boxes*5; total=total+2; print(total).

Run the instructions in this order with ordinary integer arithmetic.

Trace the book-box state changes



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Read input	boxes is assigned 3.
2. Multiply	total becomes 3×5=15.
3. Update total	total becomes its current value 15 plus 2: 17.
4. Print	The program prints 17 once.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

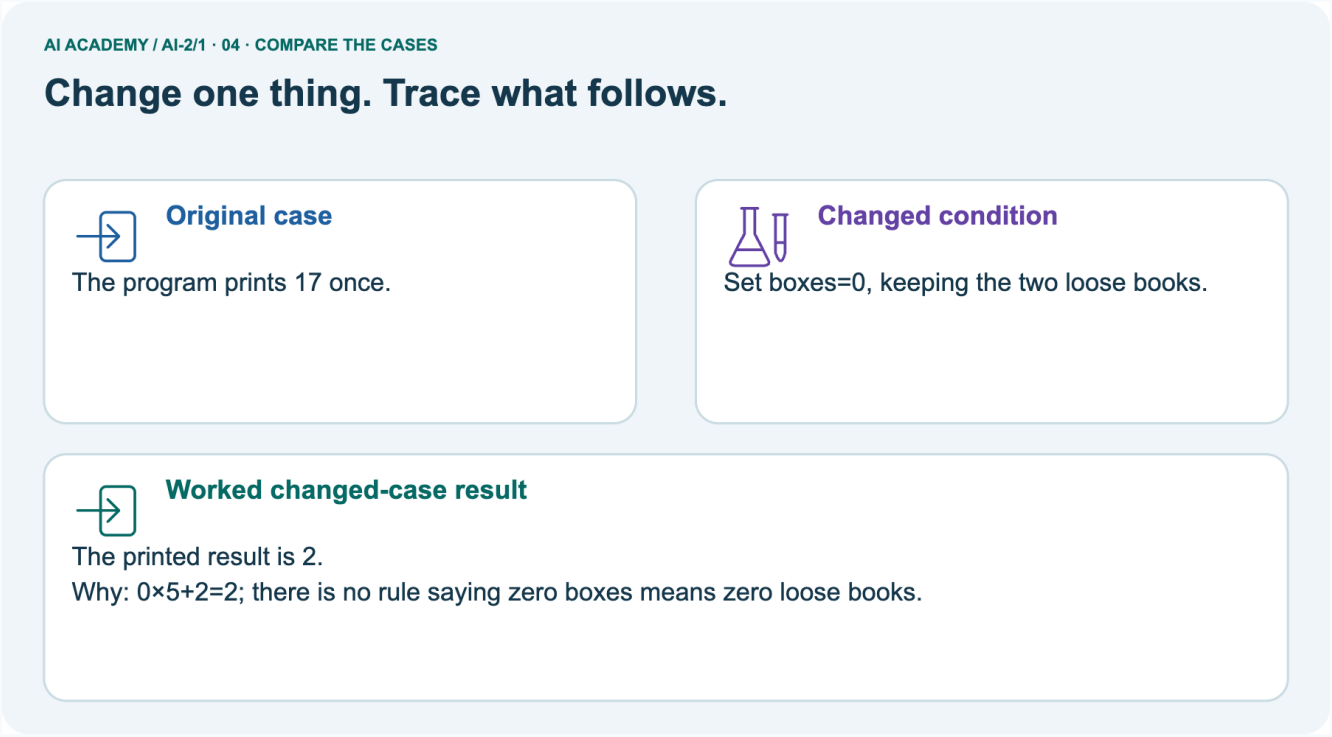
Assignment replaces a variable's stored value. In total=total+2, Python first reads the current total on the right, calculates 17, then assigns that result to total on the left.

The equal sign is an instruction here, not a claim that 15 and 17 are mathematically equal. Predict **output** before running.

Predict zero boxes and earlier printing

- Before opening the changed-box picture, set boxes to zero but keep the two loose books. Predict the final output. Then move print before the addition and explain the different value displayed.

▼ Check the changed-box calculation



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain earlier printing and the input limit

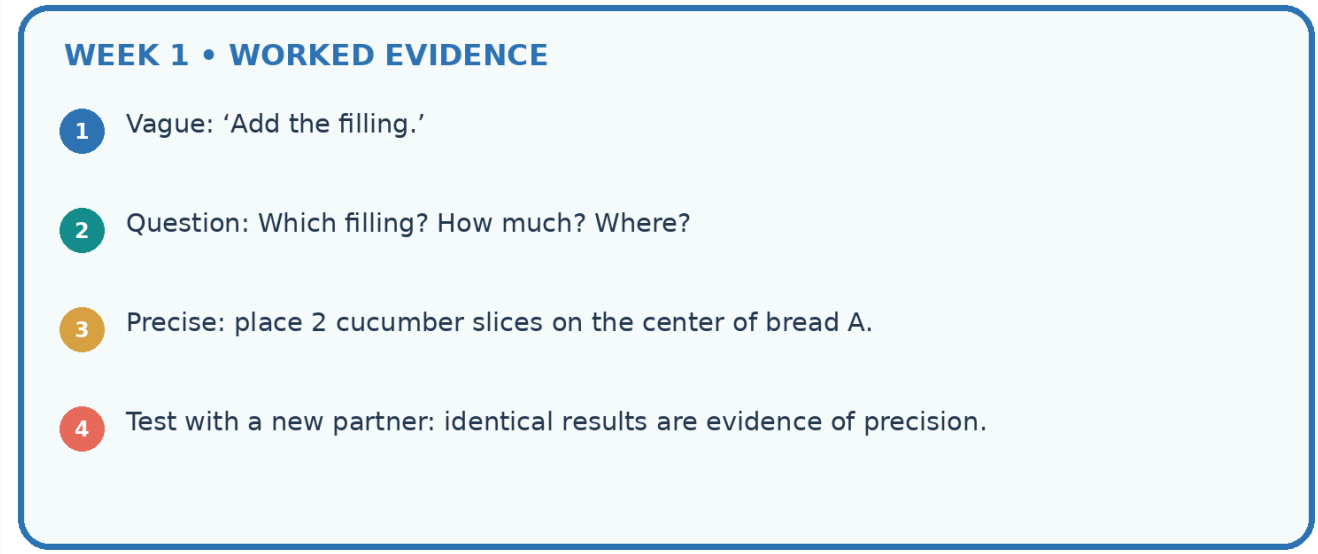
- Read the labelled picture: If print(total) moves before total=total+2, what gets printed for the original **input**?
- These statements count supplied books; they do not validate **input** types or infer stock from measurements.

▼ Earlier wording, alternatives and picture transcripts

WORKED **EVIDENCE**

Reason Step by Step

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.



1. Vague: ‘Add the filling.’

2. Question: Which filling? How much? Where?

3. **Precise**: place 2 cucumber slices on the center of bread A.

4. Test with a new partner: identical results are **evidence** of precision.

Recalculate, retrace, or restate the reasoning in your own representation:

> **Write or draw your reasoning and evidence.**

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

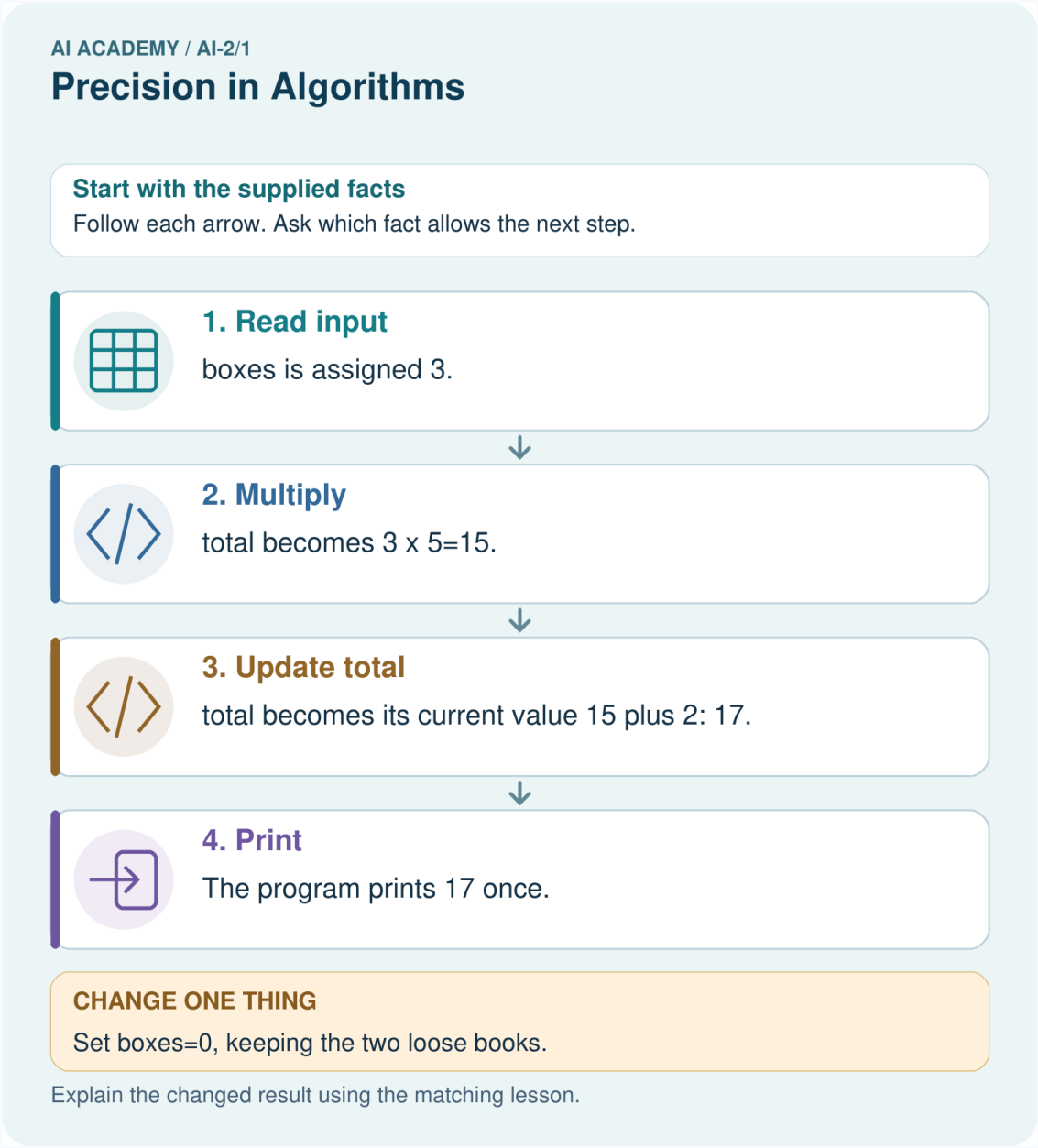
A closer explanation

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Precision in Algorithms

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Change one thing: Set boxes=0, keeping the two loose books. Predict the result before checking the explanation. What remains the same?

MAIN LESSON · SECTION 7

Debug against the requirement, not just the error message

> An error-free run can compute the wrong operation.

Keep the saved version and evidence limits

Save the file with the **input** values, expected and observed **output**s, and a short explanation. This **program** follows a written multiplication rule.

It has not trained a model, predicted somebody's preferences or checked that real packs contain the stated number. Its conclusion depends on the supplied data. Week 2 decomposes a calculation into a reusable function with explicit arguments.

Diagnose five packs incorrectly added to six

> **A script runs without an error and displays 11 for five packs of six. Is it correct for the stated requirement? Give the required result, the likely wrong operation and one check after repair. State whether your evidence came from a device run or a paper trace.**

▼ Earlier wording, alternatives and picture transcripts

ERROR DETECTIVE

Compare → Diagnose → Repair

Do not patch the answer. Find the earliest unsupported **assumption**, wrong step, or weak **evidence**.

!

COMPARE • DIAGNOSE

Weak idea: A longer instruction is automatically more precise. Strong move: locate evidence, change one cause, and retest.

Evidence record	Expected	Actual	Likely cause / next test
normal case	works		
boundary case	stated rule		
missing input	safe response		
fresh transfer	generalizes		

Repair one cause. Why is this repair better than changing several things?

Write or draw your reasoning and evidence.

A closer explanation

Practise: predict, save, run and compare

> Change one input, preserve the earlier result and explain what each line does.

Change only the number of packs

Change only packs to 5. Predict the result from five groups of six, save, then run the entire file again. The expected **output** is 30. Keep the original four-pack result beside the new one and label the changed **input**.

A comment can explain an instruction: Python ignores text after # outside a string. A comment does not change the calculation.

Run the complete five-pack script

```
packs = 5  # five equal packs
per_pack = 6
total = packs * per_pack
print(total)
```

Distinguish addition from equal-group multiplication

A **program** can run successfully and still solve the wrong problem. Replacing * with + would display 11 for this **input**, but addition does not count five groups of six.

Use the requirement and a paper calculation to check meaning; the absence of an error message is not proof of correctness.

Record W2 before checking

Use workbook W2 for the connected example “From a paper **algorithm** to Python”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Repair the assignment-displays claim

- Use the worked case for From a paper **algorithm** to Python. Proposed explanation: “An equals sign displays the answer”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the book-box print location

If print(total) moves before total=total+2, what gets printed for the original **input**?

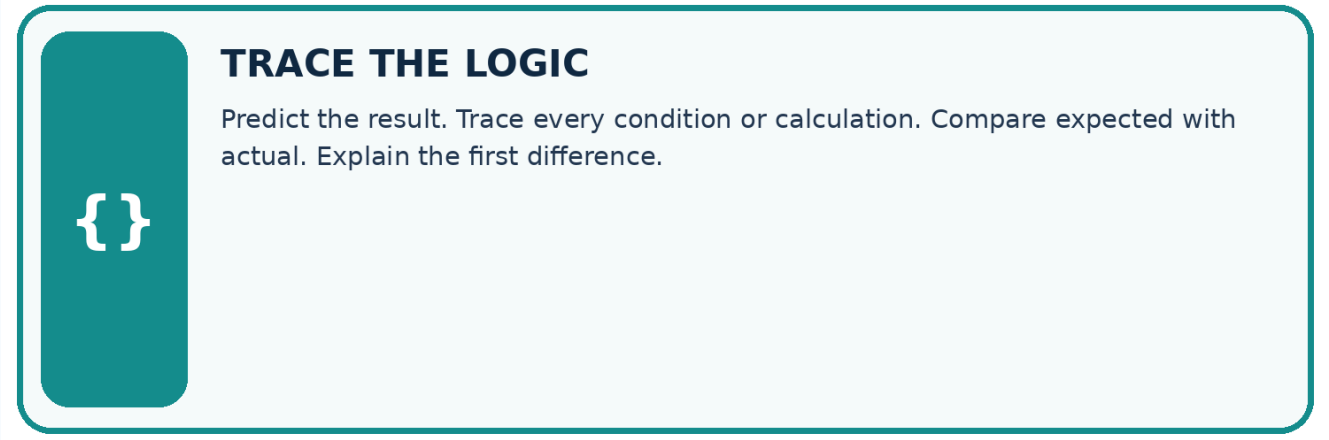
This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Earlier wording, alternatives and picture transcripts

LOGIC STUDIO

Trace Before You Run

Block code, pseudocode, tables, and diagrams are different ways to expose the same reasoning.



SET **evidence**_record PREDICT expected_result RUN one normal case and one boundary case COMPARE actual_result with expected_result IF different: locate first cause; change one factor; RETEST EXPLAIN result, limitation, and next safe step

Trace one **input**. Which line changes the state or chooses a path?

Write or draw your reasoning and evidence.

A closer explanation

Practise with less help: Predict then observe

Check your reasoning

Explain the picture

My explanation using a pictured step and the given facts

MAIN LESSON · SECTION 9

Investigate precise instructions and runnable calculations

> Use paper first, then check the explicit arithmetic program.

Prepare the two supplied stations

Two connected paper-and-code stations. First use two paper bread squares and a drawn filling to write precise sandwich instructions; a partner follows only what is written and marks ambiguities. Keep this a paper simulation. Then run the supplied counters.py script with packs=4, per_pack=6, and repeat with packs=5 and packs=0, changing only packs. Before each run predict the total using equal groups.

Predict, operate, record and repair

1. For each code run retain the saved script, expected output, actual output/error and explanation. Run the whole saved file in a fresh execution, not selected lines with leftover state. Compare a version using + instead of * for five packs of six; locate and repair the operation, then recheck the original and zero-pack cases. If no device is used, label all results paper traces. Keep the independent tile/tray task for a separate attempt.

Keep a traceable test table

Test / input · Prediction · Actual · Evidence / calculation · Change or keep?

> **Test / input:** normal
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

Test / input: boundary

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: missing

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: fresh transfer

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Reflect on the most revealing test

- What did the hardest test teach you that the normal test did not?

- Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

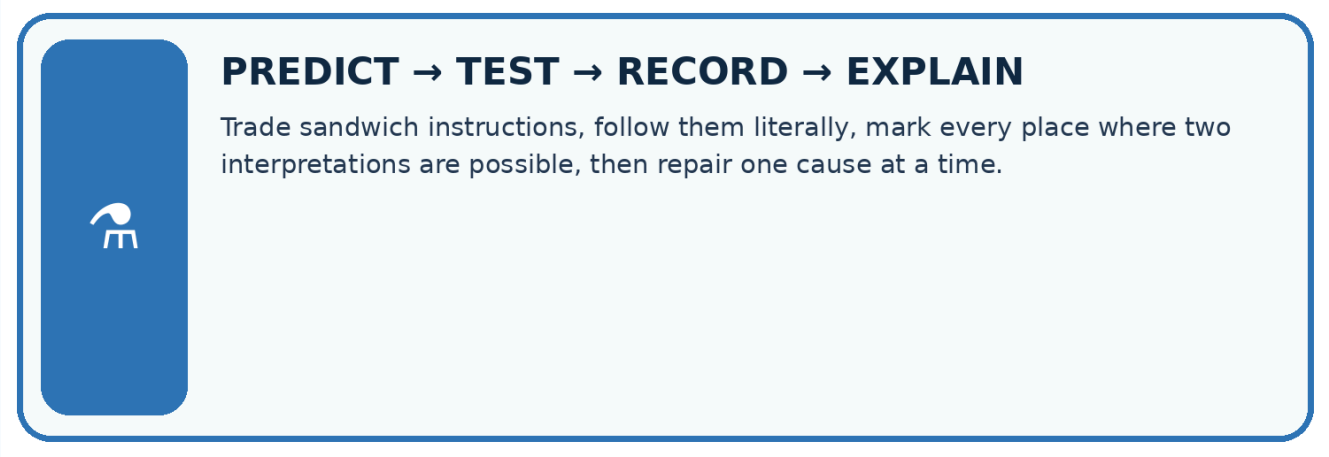
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

EVIDENCE LAB

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.



Trade sandwich instructions, follow them literally, mark every place where two interpretations are possible, then repair one cause at a time.

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: write and trace the tile script

> Use trays and per_tray before feedback.

The independent tile-count task

A fictional shelf has 3 trays of 7 tiles each. Write a four-line Python script using trays, per_tray and tiles. Predict its **output**. Then change only trays to 0 and predict again. Explain why adding the two quantities is wrong.

Keep your first attempt

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Transfer the procedure independently

Week 1 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

A robot starts at square 0. Replace move a bit and stop nearby with instructions that put it exactly on square 3 using steps of one square.

My answer, with a trace, calculation or supporting **evidence**

> **Write or draw your reasoning and evidence.**

Create and test

Change one relevant detail. Predict the result before testing. Include a boundary or missing-information case when it helps reveal a limit.

My changed **input** and expected result

> **Write or draw your reasoning and evidence.**

Connect to your project

Write exact scoring steps so two people rank the same book the same way.

The **evidence** I will keep

> **Write or draw your reasoning and evidence.**

Transfer practice from the original programme

Write an **algorithm** for borrowing a classroom book when the book may already be on loan. Include **input**s, a no-copy-available outcome and a check that prevents two borrowers receiving the same copy.

> **Explain the mechanism, evidence and one limit.**

Use feedback to improve

MAIN LESSON · SECTION 11

Begin the Transparent Book Recommender portfolio

> Carry forward precise instructions, minimal data and human choice.

What this first week contributes

This year's cumulative project is a Transparent Book Recommender. This first week contributes precise instructions and small arithmetic traces, not a finished scoring system. Use fictional books and a teacher-supplied scoring rule when one is introduced; do not invent scores or collect reader profiles to complete this checkpoint.

The system map and project evidence

THIS WEEK'S CHECKPOINT Write exact scoring steps so two people rank the same book the same way.

SYSTEM MAP Reader need → allowed **input**s → transparent rule or score → ranked candidates → explanation → test record → **human decision**

Use small predicted traces when translating the Transparent Book Recommender's paper logic into code.

Record the change and human-control boundary

- What will you add, change, or verify in the project this week?

- What should remain under **human control**?

Write or draw your reasoning and evidence.

Write or draw your reasoning and evidence.

Keep privacy and ranking limits visible

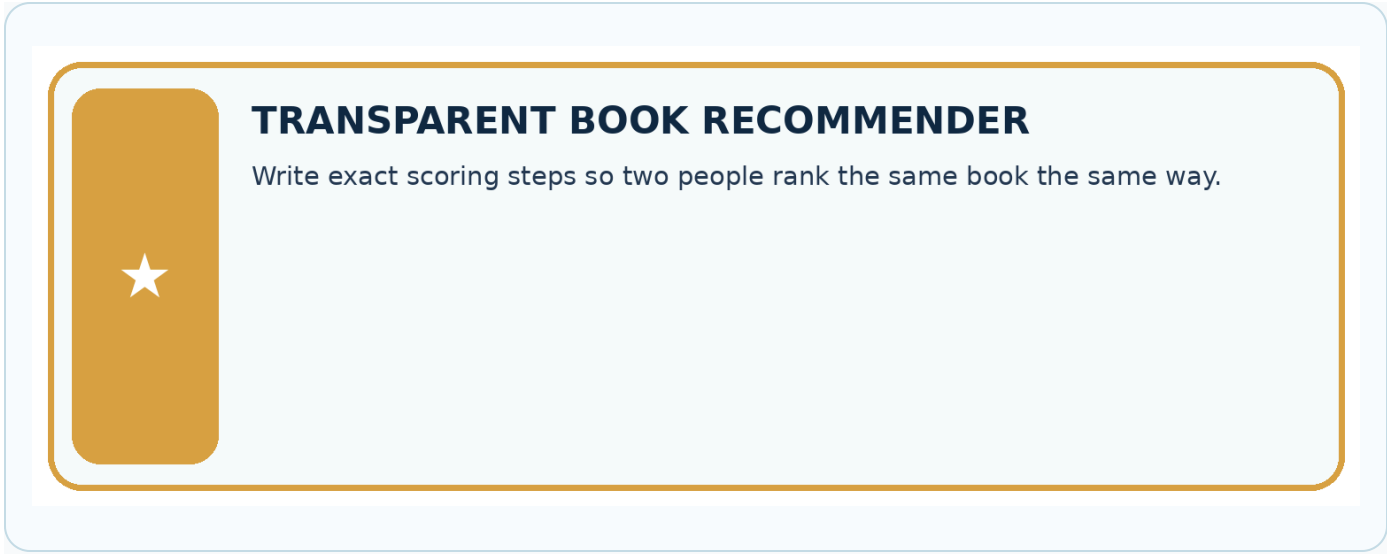
RESPONSIBLE DESIGN Collect only necessary information. Show uncertainty and limits.
Never confuse a ranking with a fact about a person.

▼ **Earlier wording, alternatives and picture transcripts**

CUMULATIVE PROJECT

Transparent Book Recommender

Every week adds one inspectable piece. The system must show how it reached a result.



Evidence for your project

MAIN LESSON · SECTION 16

Finish: explain assignment, order and displayed output

> Close your notes and distinguish a prediction from an execution.

Your exit explanation

- Close your notes. Explain this goal in your own words: Translate a short paper calculation into a Python script

My deciding step, evidence and one remaining question

Choose the next changed-case check

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or **assumption** and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Your quick word guide**

REFERENCE · SECTION 5

Your quick word guide

> Use the words to explain a particular step, record or limit.

Core vocabulary

Word	My own example	Not this / boundary
algorithm	A specified sequence of steps for performing a task or solving a class of problems.	
precise	Specific enough that a reader can distinguish the intended meaning or reproduce the action.	
assumption	Something treated as true for reasoning that has not been established by the supplied evidence.	
test case	A specified input and conditions with an expected result or measurement rule.	

Terms for the connected explanation

Word	Meaning
Program	Instructions written in a language a computer can execute

Word	Meaning
Interpreter	Software that executes Python instructions
Assignment	Binding a name to the value of an expression
Output	Information a program displays or returns

▼ Earlier wording, alternatives and picture transcripts

VOCABULARY LAB

Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.



2

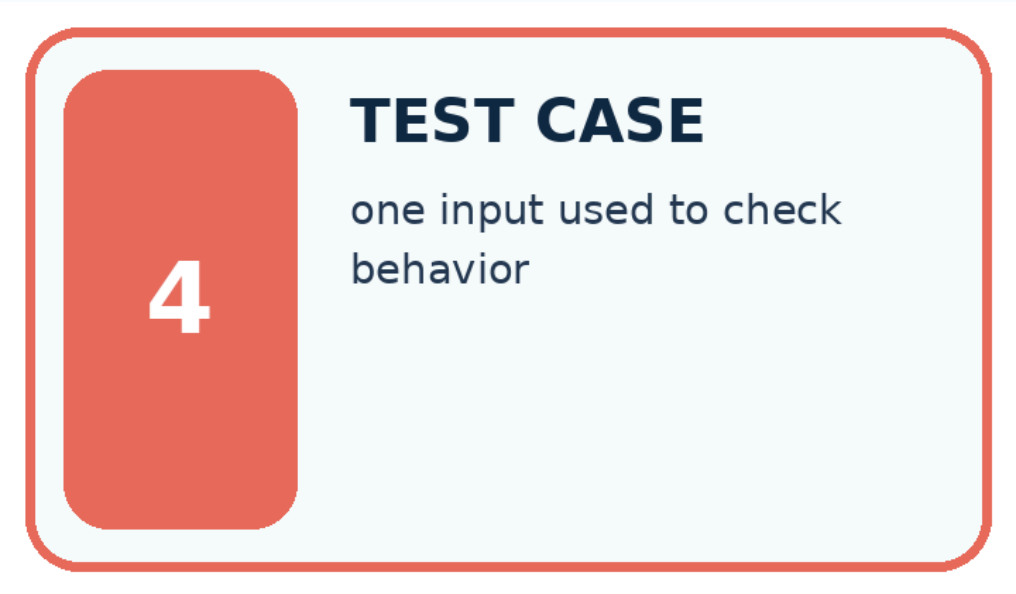
PRECISE

clear enough to repeat

3

ASSUMPTION

an unstated idea treated as true



Words used in the closer explanation

▼ **Choose a stronger thinking challenge**

REFERENCE · SECTION 10

Choose a stronger thinking challenge

> Choose the next useful challenge rather than treating the whole ladder as compulsory.

Pick your next step

- 1 • NAME: Define the concept without repeating the textbook.
- 2 • TRACE: Show the information, state, branch, calculation, or **evidence** path.
- 3 • APPLY: Solve a different example independently.
- 4 • CHALLENGE: Find a boundary case, counterexample, or unfair outcome.

- 5 • EXPLAIN LIMITS: Say when the result should not be trusted or used alone.

- My strongest **evidence** of independent understanding:

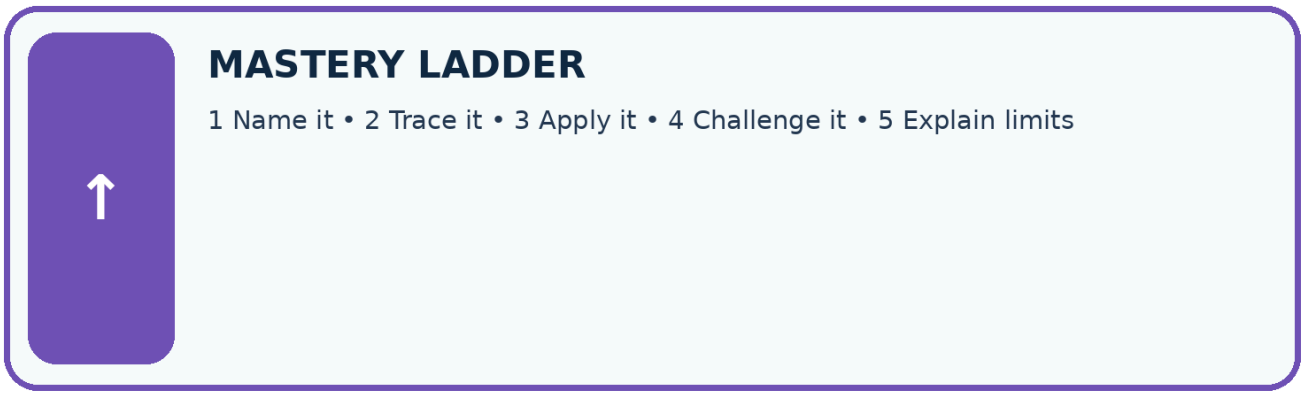
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

MASTERY

From Knowing the Word to Owning the Idea

Move upward only when you can produce **evidence** without copying the worked example.



▼ Reference: check your explanation

Reference: check your explanation

> Use after practice. Transfer the procedure independently is the selected independent case.

Check the evidence and its limit

- 1. My fresh example, trace, calculation, or design:
- 2. My **evidence** that it works (include a boundary or missing-data case):
- 3. One limitation, fairness concern, or situation needing human review:

> **Write or draw your reasoning and evidence.**

> **Write or draw your reasoning and evidence.**

Self-check and record

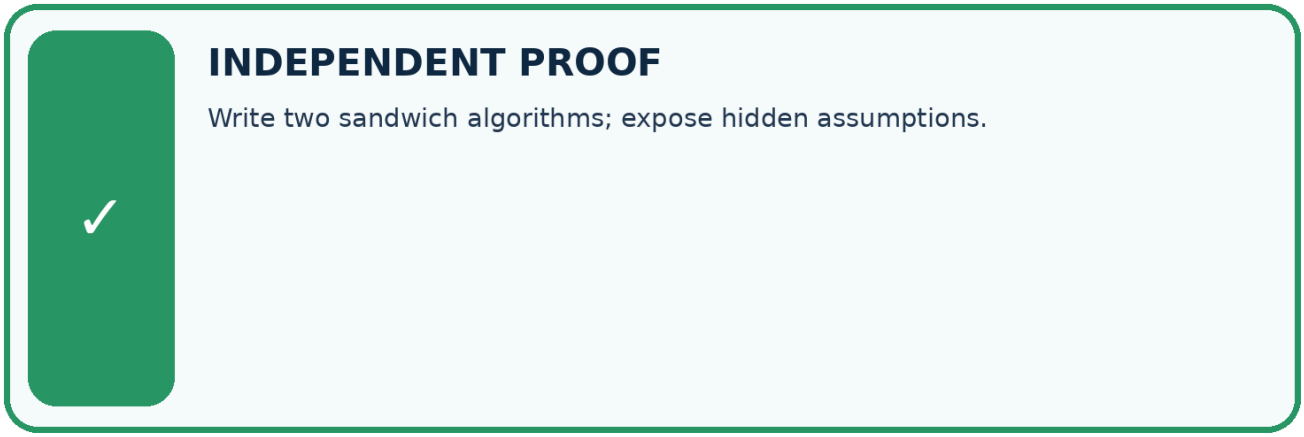
SELF-CHECK ☐ I explained the **mechanism** ☐ I used **evidence** ☐ I tested a fresh case ☐ I named a limitation ☐ I can answer 'why?'

▼ **Earlier wording, alternatives and picture transcripts**

INDEPENDENT MASTERY PROOF

Checklist for explaining this week's idea

Use after practice. Transfer the procedure independently is the selected independent case.



EXPLANATION CHECK Write ordered statements, trace current values and displayed output, and retain predictions separately from observed runs.

Write or draw your reasoning and evidence.

▼ **Extra practice: Make hidden instructions explicit**

REFERENCE · SECTION 13

Extra practice: Make hidden instructions explicit

Try the question before opening its worked explanation.

Read the supplied case and question

A sandwich **algorithm** says add spread, close it, cut it, without stating materials or directions.

Expose **assumption**s and write a **precise** paper simulation.

Your explanation

- Which detail supports the answer, and why does it matter?

Write or draw your reasoning and evidence.

▼ Check after trying

Use two paper bread squares, mark a spoon-sized spread area on one, place the second on top and draw a diagonal cut. State materials and order. No real food or sharp tools are needed.

Evidence or stage	Value or result
Materials	Two paper bread squares
Spread	Mark one spoon-sized area
Close	Place second square on top
Cut	Draw a diagonal line

▼ Earlier wording, alternatives and picture transcripts

Week 1 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

Make hidden instructions explicit

A sandwich **algorithm** says add spread, close it, cut it, without stating materials or directions.

Expose **assumption**s and write a **precise** paper simulation.

Use two paper bread squares, mark a spoon-sized spread area on one, place the second on top and draw a diagonal cut. State materials and order. No real food or sharp tools are needed.

▼ Reference: original lesson checklist

REFERENCE · SECTION 15

Reference: original lesson checklist

> Use an additional checklist when it helps inspect your explanation.

Targets and evidence

By the end, I can	How I will show it
Translate a short paper calculation into a Python script	Explain the deciding step, show a trace or test, and state one limit.
Predict and explain the order of executed statements	Explain the deciding step, show a trace or test, and state one limit.
Keep expected and observed output separate when checking a program	Explain the deciding step, show a trace or test, and state one limit.

Keep your route and first prediction

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A fictional book-box counter has three boxes with five books each and two loose books.

▼ Extra practice: pictures, repair and feedback

REFERENCE · SECTION 17

Extra practice: pictures, repair and feedback

> Attempt each question before opening its explanation. Use one browser or paper copy.

First supported question

- A fictional book-box counter has three boxes with five books each and two loose books.
- Which labelled step matches this detail? The **program** prints 17 once.
- 1. Read **input**
- 2. Print
- 3. Multiply
- 4. Update total
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ Check the first explanation

After your attempt, compare with the supported explanation: Print The **program** prints 17 once. **Assignment** replaces a variable's stored value.

In total=total+2, Python first reads the current total on the right, calculates 17, then assigns that result to total on the left. The equal sign is an instruction here, not a claim that 15 and 17 are mathematically equal.

Predict **output** before running.

Changed-case question

- Changed-case practice: Set boxes=0, keeping the two loose books. Which conclusion is supported by the supplied facts?
- 1. This one example guarantees correct results for every future **input**.
- 2. The printed result is 2.
- 3. We can decide the result without examining the changed **input** or the procedure.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed case**

After your attempt, compare with the supported explanation: The printed result is 2.
0×5+2=2; there is no rule saying zero boxes means zero loose books.

Repair and save your work

Review Expo and Defense (ai-1/36). Goal: Evaluate a saved candidate and a manual baseline fairly. Short practice: Write one defensible success statement and one claim the **evidence** cannot support..

Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work
</details>

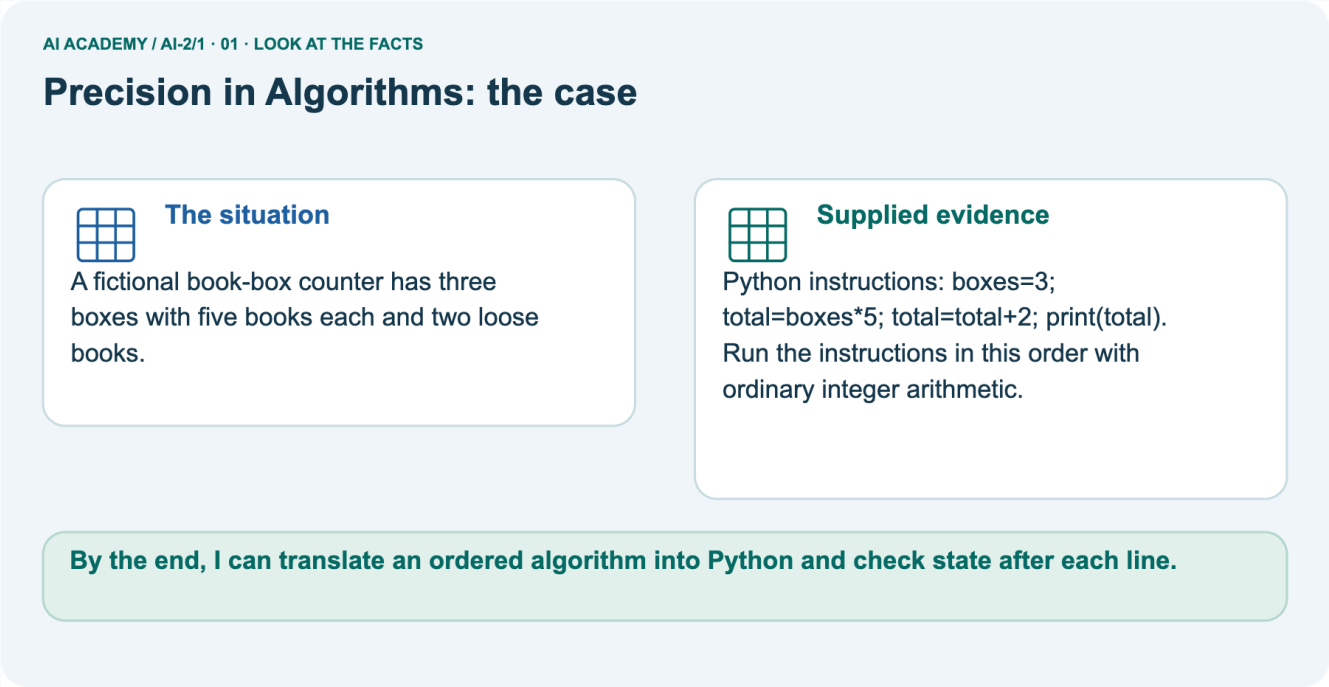
<details>
<summary>Reference: the six-picture journey</summary>
</details>

REFERENCE · SECTION 18

Reference: the six-picture journey

> Revisit the picture that serves the current question; key diagrams already sit beside the main explanation.

01. Precision in Algorithms: the case



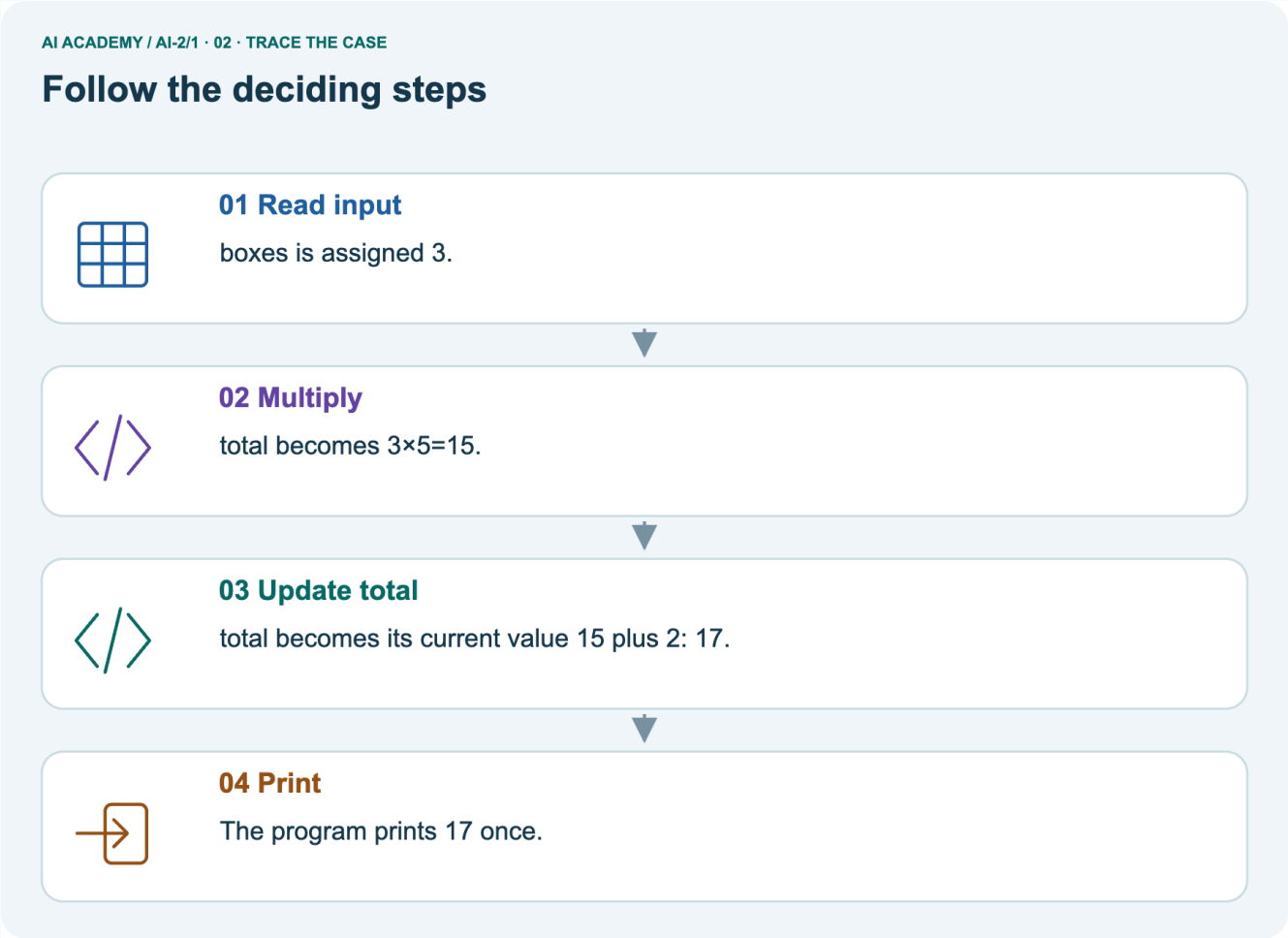
Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional book-box counter has three boxes with five books each and two loose books.
Supplied facts: Python instructions: boxes=3; total=boxes*5; total=total+2; print(total).

Run the instructions in this order with ordinary integer arithmetic.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Read **input**: boxes is assigned 3. → Multiply: total becomes 3×5=15. → Update total: total becomes its current value 15 plus 2: 17. → Print: The **program** prints 17 once.

Point to each numbered stage and explain which recorded **input** it uses.

03. A calculation from paper to output

AI ACADEMY / AI-2/1 · 03 · INSPECT THE LESSON MODEL

A calculation from paper to output

Paper step	Python instruction	Effect
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

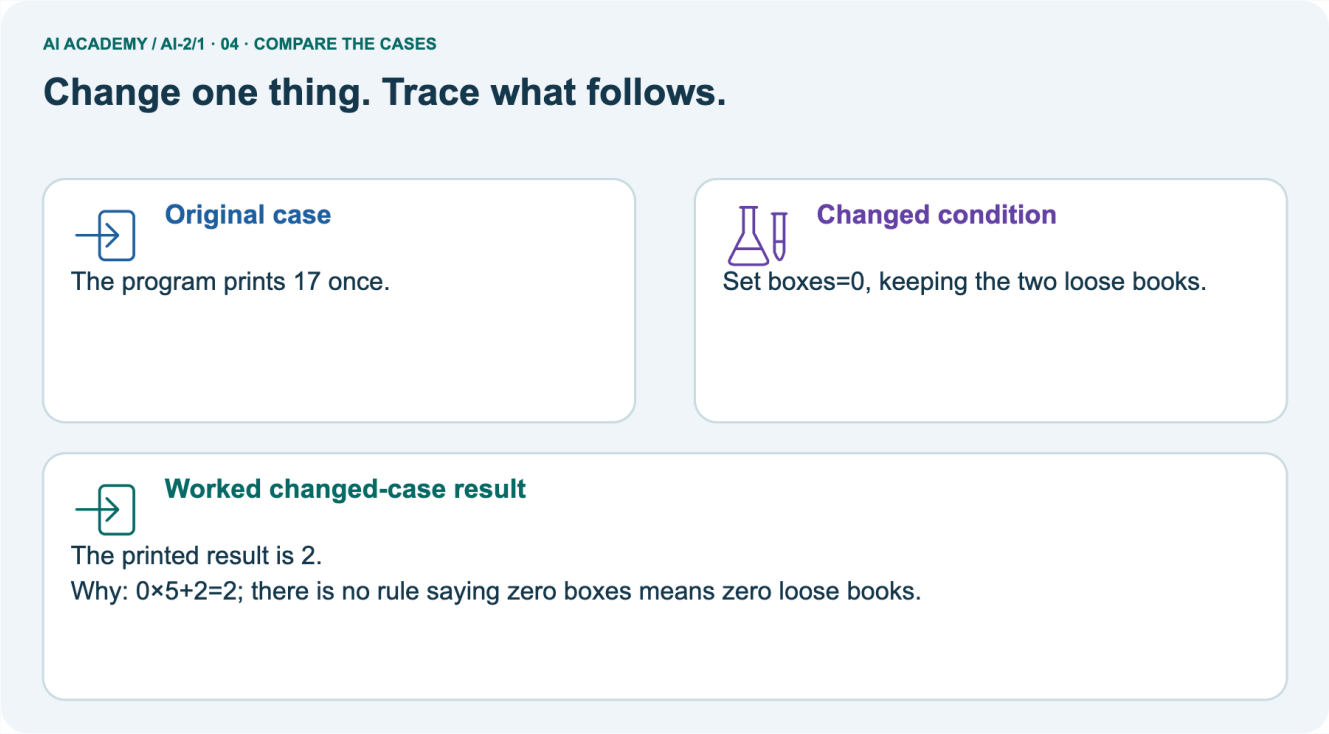
Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

Follow the labelled picture with the matching explanation. Open the vector diagram

A calculation from paper to **output**. Record pack count | packs = 4 | Name holds 4. Multiply two quantities | total = packs * per_pack | Name holds product. Show the answer | print(total) | Displays the value

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.

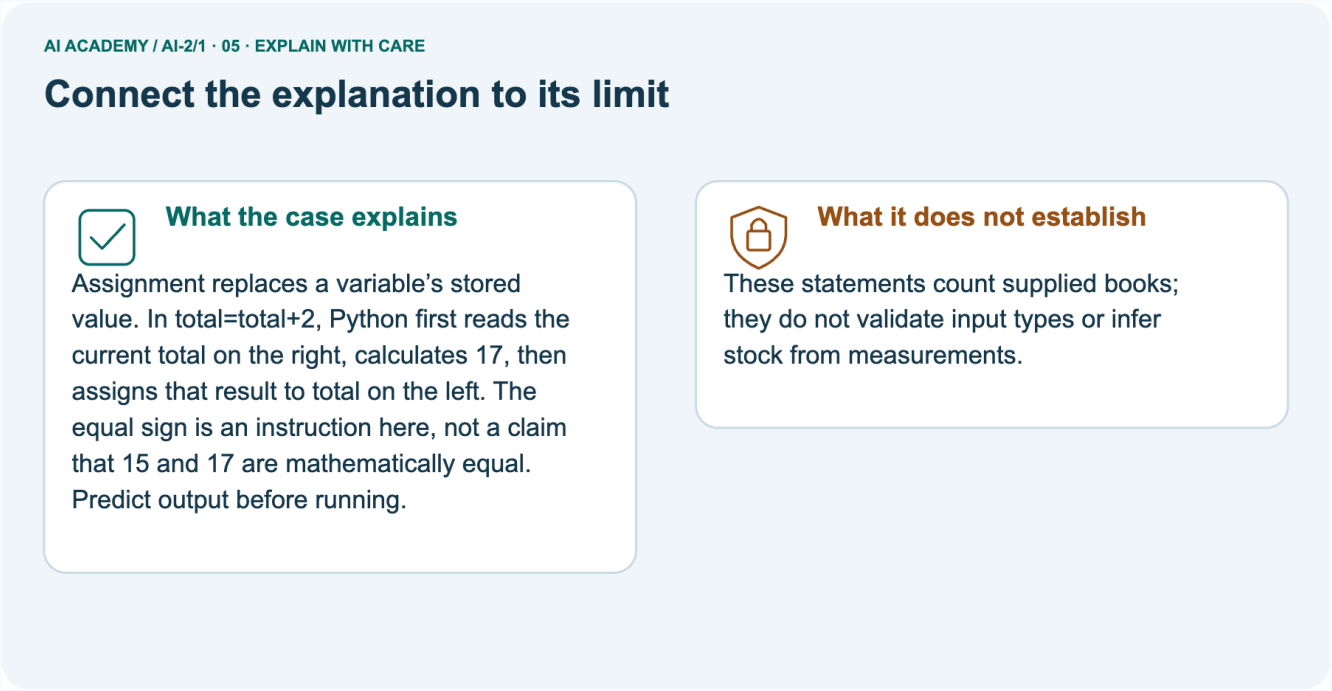


Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: The **program** prints 17 once. Changed condition: Set boxes=0, keeping the two loose books. Expected result: The printed result is 2. Reason: 0×5+2=2; there is no rule saying zero boxes means zero loose books.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

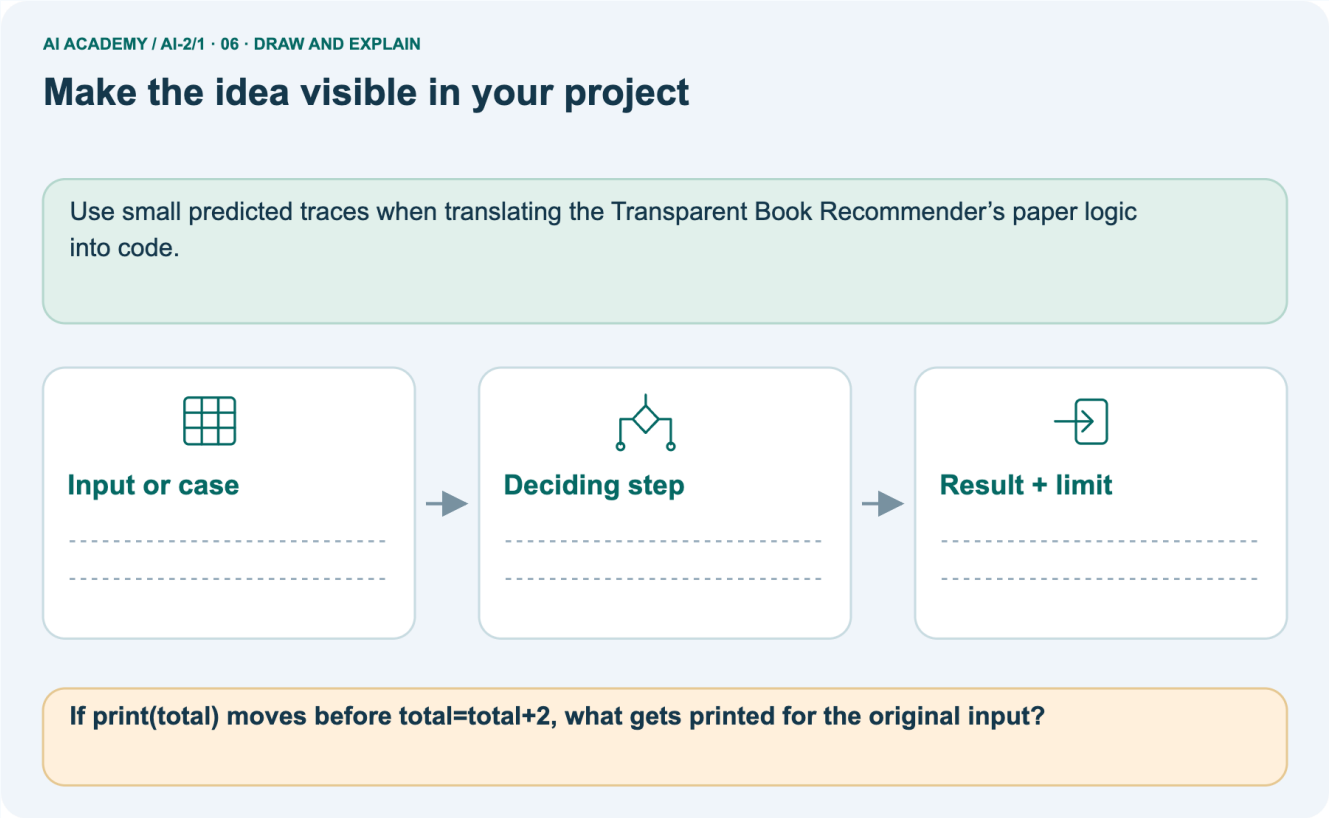
Assignment replaces a variable's stored value. In total=total+2, Python first reads the current total on the right, calculates 17, then assigns that result to total on the left.

The equal sign is an instruction here, not a claim that 15 and 17 are mathematically equal. Predict **output** before running.

Limit: These statements count supplied books; they do not validate **input** types or infer stock from measurements.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

Use small predicted traces when translating the Transparent Book Recommender's paper logic into code. Draw your **input**, deciding step, result and limit.

Supported question: If print(total) moves before total=total+2, what gets printed for the original **input**?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Precision in Algorithms: the case

02. Follow the deciding steps

03. A calculation from paper to output

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.